

Unidade I

No C# 14, ao atribuir uma variável de um tipo declarado com class para outra variável do mesmo tipo, o que a atribuição copia?

Resposta Seleccionada:



b.

Ela copia a referência para o mesmo objeto.

No site oficial da Microsoft, quais são as três edições do Visual Studio 2026?

Resposta Seleccionada:



a.

A edição Community, a edição Professional e a edição Enterprise.

Qual alternativa descreve corretamente como os quatro pilares da POO, em conjunto, orientam um projeto em C#?

Resposta
Seleccionada:



b.

Eles definem como os tipos são organizados, como o acesso a detalhes internos é limitado, como comportamentos são reutilizados entre classes e como objetos diferentes podem responder de modo distinto à mesma operação.

Qual alternativa descreve corretamente a relação entre os invariantes de classe, o encapsulamento e os objetos imutáveis em C#?

Resposta
Seleccionada:



c.

O encapsulamento permite centralizar e controlar mudanças no estado, ajudando a preservar invariantes; e objetos imutáveis simplificam essa preservação porque, uma vez construídos corretamente, não mudam mais de estado.

Quest I

Pergunta 1

0,25 em 0,25 pontos



Em um projeto de console criado com os modelos atuais do .NET, considere o arquivo Program.cs com apenas as linhas abaixo:

```
// Programa "Olá, Mundo" utilizando instruções de nível superior em C#  
  
Console.WriteLine("Olá, Mundo!");
```

Qual alternativa descreve corretamente o que precisa ser verdade para esse código compilar e o que o compilador produz como ponto de entrada do executável?

Resposta

Selecionada:



b.

A compilação é viável sem using System; porque diretivas using implícitas já colocam System no escopo; o compilador sintetiza um método de ponto de entrada e uma estrutura de suporte para encapsular as instruções de nível superior.

Pergunta 2

0,25 em 0,25 pontos



2- Você está estruturando uma solução com múltiplos projetos (src/, tests/ etc.) e quer atingir simultaneamente três objetivos: (1) manter o grafo de dependências entre módulos explícito e integrado ao build, (2) unificar versões de pacotes externos para evitar divergências entre projetos, e (3) aplicar políticas transversais (por exemplo, nulabilidade e usings implícitos) de forma consistente. Qual alternativa descreve uma configuração coerente com esses objetivos?

Resposta
Selecionada:

✓ b.

Usar ProjectReference para dependências internas entre .csproj, centralizar versões em Directory.Packages.props na raiz (habilitando ManagePackageVersionsCentrally e declarando PackageVersion), deixar os projetos com PackageReference sem Version, e aplicar políticas comuns via Directory.Build.props e regras de estilo via .editorconfig.

Pergunta 3

0,25 em 0,25 pontos



Em C# 14, em um programa com instruções de nível superior (top-level statements), considere a linha cliente?.Total += 100m;, em que cliente tem o tipo Cliente? e Total é uma propriedade decimal com get; set;. Qual alternativa descreve corretamente o comportamento dessa linha, incluindo o caso em que cliente seja null?

Resposta
Selecionada:

✓ c.

A expressão não executa nenhuma leitura nem escrita se cliente for null; se cliente não for null, ela lê Total, soma 100m e grava o resultado de volta em Total

Pergunta 4

0,25 em 0,25 pontos



Considere o tipo public record Produto(int Id, string Nome, decimal Preco); e o uso: var p2 = p1 with { Preco = 18m };. Qual alternativa está correta sobre a identidade das instâncias, a igualdade e o efeito do with, assumindo que p1 foi criado como new Produto(1, "Café", 15m)?

Resposta
Selecionada:

✓ c.

A expressão with cria uma nova instância, preserva p1 inalterado e faz com que p1 e p2 sejam diferentes em igualdade estrutural, pois diferem em Preco.

Pergunta 5

0,25 em 0,25 pontos



Você quer um projeto de console em estilo SDK (SDK-style) voltado a net10.0, com referências anuláveis habilitadas, ImplicitUsings habilitado e sem geração automática de atributos de assembly (ou seja, com controle manual via código, quando desejado). Qual arquivo .csproj atende corretamente a esses requisitos com a configuração essencial, sem depender de listagem manual de arquivos .cs?

Resposta Selecionada:  a.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>

    <GenerateAssemblyInfo>false</GenerateAssemblyInfo>
  </PropertyGroup>
</Project>
```

Pergunta 6

0,25 em 0,25 pontos



Em um trecho de código em C#, considere a sequência abaixo:

```
using System;
using System.Collections.Generic;

class Programa
{
    static void Main()
    {
        List<double> constants = [ Math.PI, Math.E ];
        System.Console.WriteLine(constants.Count);
    }
}
```

Qual é o papel do ponto nas diferentes ocorrências?

Resposta

Selecionada:



b.

O ponto forma nomes qualificados em `System.Console` e em `System.Collections.Generic`, e também realiza acesso a membros; em `Math.PI` e em `Math.E`, o acesso é a membros estáticos, e, em `constants.Count`, o acesso é a membro de instância; além disso, expressões do tipo `e.M` e invocações do tipo `e.M(e1, ..., ev)` integram o conjunto de expressões de acesso a membro que o compilador precisa vincular a um membro concreto durante a resolução.

pergunta 7

0,25 em 0,25 pontos



Você precisa redesenhar uma classe de conta bancária em C# com as seguintes exigências simultâneas: (i) o código externo não pode alterar o saldo de maneira arbitrária; (ii) depósitos com quantia não positiva devem falhar; (iii) saques com quantia não positiva ou acima do saldo devem falhar; (iv) o saldo deve poder ser consultado externamente; (v) a solução deve reduzir o risco de estados inválidos e evitar dependências do código cliente em detalhes internos. Qual das alternativas abaixo corresponde ao código que melhor atende a essas exigências?

Resposta
Selecionada:

✓ c.

```
public class ContaBancaria
{
    private decimal saldo;
    public decimal Saldo
    {
        get { return saldo; }
        private set { saldo = value; }
    }
    public ContaBancaria(decimal saldoInicial = 0m)
    {
        if (saldoInicial < 0m)
            throw new
ArgumentOutOfRangeException(nameof(saldoInicial));
        saldo = saldoInicial;
    }
    public bool Depositar(decimal quantia)
    {
        if (quantia <= 0)
            return false;
        saldo += quantia;
        return true;
    }
    public bool Sacar(decimal quantia)
    {
        if (quantia <= 0 || quantia > saldo)
            return false;
        saldo -= quantia;
        return true;
    }
}
```

Pergunta 8

0,25 em 0,25 pontos



Em uma classe Pessoa que encapsula a idade, decide-se alterar a representação interna de “anos” para “meses”, mantendo inalterado o contrato público de Idade (continuar expondo anos) e preservando o invariante Idade ≥ 0 . Qual alternativa descreve a refatoração que melhor preserva o encapsulamento, mantém o invariante centralizado e reduz o risco de inconsistências internas?

Resposta
Selecionada:



c.
A substituição do campo privado por `private int idadeEmMeses;`, com a propriedade `Idade` convertendo anos↔meses, validando no set e armazenando sempre meses no campo privado; além disso, os métodos internos que alteram a idade passam a fazê-lo por meio da propriedade (quando a lógica da propriedade for relevante).

Pergunta 9

0,25 em 0,25 pontos



Considere uma classe que impõe o invariante $\text{idade} \geq 0$ por meio de um campo privado e de uma propriedade `Idade` com validação no set. Suponha que um método público precise recalcular a idade a partir de múltiplos parâmetros e possa, durante o cálculo, obter valores intermediários negativos. À luz da noção de invariante (válido antes e depois de chamadas a métodos públicos, podendo ser quebrado temporariamente apenas durante a execução interna), qual alternativa descreve uma implementação que respeita o invariante “em todos os momentos observáveis externamente”?

Resposta
Selecionada:



c.
O método mantém todos os valores intermediários em variáveis locais, valida o valor final e realiza uma única atribuição ao estado do objeto (via propriedade, quando a lógica de validação for parte do contrato), assegurando que nenhuma interação externa aconteça enquanto o valor intermediário estiver inválido.

Pergunta 10

0,25 em 0,25 pontos



Analise o código a seguir. Considere que ele está distribuído em dois arquivos na mesma solução:

Arquivo Pessoa.cs:

```
public record Pessoa(string Nome, int Idade);
```

Arquivo Program.cs (com instruções de nível superior):

```
var p = new Pessoa("Ana", 30);  
var p2 = p with { Idade = 31 };  
var p3 = p with { Idade = 30 };
```

Qual alternativa está correta sobre (i) o que pode ou não compilar envolvendo init, (ii) o efeito de with e (iii) a semântica de igualdade de record?

Resposta

Selecionada:



c.

A atribuição `p.Idade = 31`; falha em compilação; `p2` e `p3` são novas instâncias; além disso, `p3.Equals(p)` retorna `true`, embora `object.ReferenceEquals(p3, p)` retorne `false`.

Unidade II

Qual alternativa descreve corretamente a diferença central entre uma classe abstrata e uma interface no design em C#?

Resposta

Selecionada:



c.

Uma classe abstrata tende a representar uma generalização em uma hierarquia de herança e pode oferecer implementação comum, enquanto uma interface define capacidades que podem ser implementadas por classes diferentes, mesmo fora da mesma hierarquia.

Qual afirmação expressa corretamente o Princípio da Substituição de Liskov (LSP)?

Resposta

Selecionada:



b.

Se `S` é um subtipo de `T`, então objetos de `T` podem ser substituídos por objetos de `S` sem quebrar o comportamento correto do programa, desde que `S` preserve o contrato esperado por `T`.

Pergunta 3

0 em 0 pontos



Qual alternativa descreve corretamente quando a herança com uma classe abstrata tende a ser apropriada?

Resposta
Selecionada:

✓ b.

Quando se quer modelar uma relação do tipo é-um, reutilizar comportamento comum, e impor que subclasses implementem certas operações por meio de métodos abstratos.

Pergunta 4

0 em 0 pontos



Qual alternativa descreve corretamente o polimorfismo em C# e as formas principais pelas quais ele aparece?

Resposta
Selecionada:

✓ d.

Ele permite tratar objetos diferentes de forma uniforme quando compartilham um tipo base ou contrato, e aparece principalmente por métodos virtuais ou abstratos em hierarquias e por interfaces, com decisão do método em tempo de execução.

Quest II

1- Considere o código abaixo:

```
using System;
```

```
// Classe abstrata base
```

```
public abstract class Forma
```

```
{
```

```
    public string Nome { get; protected set; }
```

```
    // Construtor da classe base abstrata
```

```
    protected Forma(string nome) {
```

```
        Nome = nome;
```

```
    }
```

```
    // Membro abstrato: nenhuma implementação aqui
```

```
    public abstract double CalcularArea();
```

```
    // Método concreto: implementado na classe abstrata
```

```
    public void Descrever()
```

```
    {
```

```
        Console.WriteLine($"Esta forma é um {Nome}.");
```

```
    }
```

```
}
```

```
// Subclasse concreta da forma
```

```
public class Circulo : Forma
```

```
{
```

```
    private double raio;
```

```
    public Circulo(double raio) : base("círculo") {
```

```
        this.raio = raio;
```

```
    }
```

```
    // Implementação obrigatória do método abstrato
```

```
    public override double CalcularArea()
```

```
    {
```

```
        return Math.PI * raio * raio;
```

```
    }
```

```
}
```

```
// Outra subclasse concreta
```

```
public class Retangulo : Forma
```

```
{
```

```
    private double largura, altura;
```

```
    public Retangulo(double largura, double altura) : base("retângulo") {
```

```
        this.largura = largura;
```

```
        this.altura = altura;
```

```
    }
```

```
    public override double CalcularArea()
```

```
    {
```

```
        return largura * altura;
```

```
}  
}
```

Você precisa somar áreas em um algoritmo que deve aceitar (i) objetos que reaproveitam o estado comum e o método Descrever() e (ii) objetos que não podem herdar de Forma (porque já herdam de outra classe), mas, ainda assim, devem participar do mesmo polimorfismo. Qual mudança atende a esse cenário, mantendo o contrato de “toda figura sabe calcular área” verificado em compilação e sem exigir duplicação do código compartilhado de Forma?

Resposta
Selecionada:

✔ b.

Introduzir uma interface IForma com double CalcularArea();, fazer Forma : IForma e manter CalcularArea() como abstrato em Forma, passando o algoritmo a operar sobre IForma.

Pergunta 2

0,25 em 0,25 pontos



Um cliente recebe um objeto por um tipo base e assume que pode ajustar duas dimensões de forma independente antes de calcular área. Em uma tentativa de representar um quadrado como subtipo de retângulo, o tipo derivado força as duas dimensões a permanecerem iguais, alterando o efeito de atribuições separadas. Qual opção corrige o problema de substituição sem impor ao cliente do tipo base novas restrições silenciosas?

Resposta
Selecionada:

✔ d.

Remover a relação de herança entre quadrado e retângulo e fazer ambos aderirem a um contrato comum (por exemplo, uma interface de cálculo de área), reservando a herança de classes para reuso que não altere expectativas do cliente.

Pergunta 3

0,25 em 0,25 pontos



Considere exatamente o código C# a seguir, distribuído em dois arquivos na mesma solução:

Arquivo CumprimentoMulti.cs:

```
interface IGreetingsEn {  
    string Saudar();  
}  
interface IGreetingsFr {  
    string Saudar();  
}  
  
class CumprimentoMulti : IGreetingsEn, IGreetingsFr {  
    // Implementação implícita para inglês:  
    public string Saudar() {  
        return "Hello"; // Saudação em inglês por padrão  
    }  
    // Implementação explícita para francês:  
    string IGreetingsFr.Saudar() {  
        return "Bonjour"; // Saudação em francês somente via interface IGreetingsFr  
    }  
}
```

Arquivo Program.cs (com instruções de nível superior):

```
using System;  
  
var obj = new CumprimentoMulti();  
Console.WriteLine(obj.Saudar()); // Saída: Hello  
Console.WriteLine(((IGreetingsEn)obj).Saudar()); // Saída: Hello (IGreetingsEn usa implícita)  
Console.WriteLine(((IGreetingsFr)obj).Saudar()); // Saída: Bonjour (IGreetingsFr usa explícita)
```

Qual alternativa descreve corretamente, ao mesmo tempo, (i) a vinculação em tempo de compilação do método chamado em `obj.Saudar()` e (ii) a razão pela qual a saudação em francês exige conversão explícita para a interface correspondente?

Resposta

Selecionada:



b.

A chamada `obj.Saudar()` é vinculada ao método público da classe, porque o membro explícito não integra a superfície pública do tipo concreto; a conversão é exigida porque o membro explícito só é acessível quando a referência está tipada como `IGreetingsFr`.

Pergunta 4

0,25 em 0,25 pontos



Suponha uma interface IRepository cujo contrato, na documentação, promete: “ao chamar Remove(item), o item deve ser removido se ele existir no repositório”. Sem alterar a assinatura do método nem o tipo de retorno, qual alternativa descreve uma implementação que viola a substituíbilidade exigida pelo LSP (Princípio da Substituição de Liskov), mesmo que o código compile?

Resposta
Selecionada:



b.

A implementação lança uma exceção NotSupportedException para qualquer chamada, inclusive quando o item existe; dessa forma, a remoção prometida pelo contrato deixa de acontecer.

Pergunta 5

0,25 em 0,25 pontos



Considere o código C# a seguir, distribuído em dois arquivos na mesma solução:

Arquivo Retangulo.cs:

```
class Retangulo {
    public virtual void DefinirLargura(double larg) { Largura = larg; }
    public virtual void DefinirAltura(double alt) { Altura = alt; }
    public double Largura { get; protected set; }
    public double Altura { get; protected set; }
    public virtual double Area => Largura * Altura;
}

class Quadrado : Retangulo {
    public override void DefinirLargura(double larg) {
        base.DefinirLargura(larg);
        base.DefinirAltura(larg);
    }
    public override void DefinirAltura(double alt) {
        base.DefinirAltura(alt);
        base.DefinirLargura(alt);
    }
}
```

Arquivo Program.cs (com instruções de nível superior):

```
using System;
```

```
void AumentarRetanguloEmDobro(Retangulo r) {
    r.DefinirLargura(r.Largura * 2);
    r.DefinirAltura(r.Altura * 2);
    // Esperamos que a área agora seja 4 vezes maior:
    Console.WriteLine("Área quadruplicada: " + r.Area);
}
```

```
// Cenário de uso:
```

```
Retangulo meuRet = new Retangulo();
```

```
meuRet.DefinirLargura(5);
```

```
meuRet.DefinirAltura(10);
```

```
// Área inicial = 50
```

```
AumentarRetanguloEmDobro(meuRet);
```

```
// Área esperada = 200 (50 * 4) -> correto para Retangulo
```

```
Retangulo meuQuad = new Quadrado();
```

```
meuQuad.DefinirLargura(5);
```

```
meuQuad.DefinirAltura(10);
```

```
// Aqui, ao definir altura para 10, a largura também se torna 10 (Quadrado impõe lados iguais)
```

```
// Área inicial de meuQuad = 100 (10*10)
```

```
AumentarRetanguloEmDobro(meuQuad);
```

Mantendo a lógica do código como está, qual é o valor numérico efetivamente impresso na segunda chamada de `AumentarRetanguloEmDobro(meuQuad)` (isto é, o valor de `r.Area` naquele `Console.WriteLine`)?

Resposta

Selecionada:

 d.

Ele é 1600, pois as duas chamadas (dobrar a largura e depois dobrar a altura) acabam dobrando duas vezes o lado do quadrado, levando-o a 40×40.

Pergunta 6

0,25 em 0,25 pontos



Em um jogo, existe uma classe abstrata `Personagem` com métodos concretos que mantêm invariantes (por exemplo, “a vida nunca pode ficar negativa” e “o estado de morte só é acionado uma vez”). O método público `ReceberDano(int)` é virtual, e várias subclasses o sobrescrevem, algumas chamando `base.ReceberDano(dano)` e outras não. Após uma alteração pequena na superclasse (para registrar telemetria e ajustar a ordem de atualização do estado), surgem regressões em subclasses que antes funcionavam. Qual mudança de projeto tende a reduzir de forma mais direta esse tipo de regressão em cascata, sem eliminar a reutilização de código comum?

Resposta

Selecionada:

 b.

A decisão de transformar `ReceberDano(int)` em um método não virtual que aplica o algoritmo completo e delega a variação a um ou mais ganchos `protected` (`abstract` ou `virtual`) para etapas específicas, evitando que subclasses alterem o fluxo principal.

Pergunta 7

0,25 em 0,25 pontos



Considere o código C# a seguir:

```
using System;

abstract class Personagem {
    public virtual void Falar() {
        Console.WriteLine("Base");
    }
}

class Guerreiro : Personagem {
    public sealed override void Falar() {
        Console.WriteLine("Guerreiro");
    }
}

class Paladino : Guerreiro {
    public new void Falar() {
        Console.WriteLine("Paladino");
    }
}

class Program {
    static void Main() {
        Personagem p = new Paladino();
        p.Falar();

        Paladino q = new Paladino();
        q.Falar();
    }
}
```

Qual alternativa descreve corretamente o comportamento do programa e o motivo técnico relevante?

Resposta

Selecionada:

☒ b.

O código compila, imprime “Guerreiro” e depois “Paladino”, porque a chamada via Personagem usa despacho virtual até o último override permitido (selado em Guerreiro), enquanto a chamada via Paladino resolve para o método ocultador definido com new.

Pergunta 8

0,25 em 0,25 pontos



Em um jogo, personagens podem combinar capacidades ortogonais (por exemplo, “voar”, “atacar a distância”, “ser invisível”, “aplicar veneno”), e essas capacidades podem ser ganhas ou perdidas durante a partida. Há também a restrição de que uma classe concreta só pode herdar de uma única superclasse concreta. Qual proposta de modelagem tende a evitar a explosão combinatória de subclasses e a permitir recombinações em tempo de execução com menor acoplamento estrutural?

Resposta

Selecionada:



c.

A decisão de representar cada capacidade como um componente (objeto) com uma interface estável, e de fazer a entidade do jogo manter um conjunto desses componentes, delegando a eles o comportamento conforme forem anexados ou removidos.

Pergunta 9

0,25 em 0,25 pontos



Considere o código a seguir:
using System;

```
class Character {
    public string Name { get; set; }
    public Character(string name) {
        Name = name;
    }
    public virtual void Attack() {
        Console.WriteLine($"{Name} ataca de forma genérica.");
    }
}

class Warrior : Character {
    public Warrior(string name) : base(name) { }
    public override void Attack() {
        Console.WriteLine($"{Name} avança com sua espada, desferindo um golpe poderoso!");
    }
}

class Mage : Character {
    public Mage(string name) : base(name) { }
    public override void Attack() {
        Console.WriteLine($"{Name} lança uma bola de fogo ardente contra o inimigo!");
    }
}

// Código de demonstração
class Program {
    static void Main() {
        Character[] party = new Character[2];
        party[0] = new Warrior("Arus");
        party[1] = new Mage("Luna");
        foreach (Character member in party) {
            member.Attack();
        }
    }
}
```

Agora, suponha que a palavra-chave virtual seja removida do método Attack() na classe Character, mantendo-se o restante do código exatamente como está. Qual alternativa descreve corretamente o que acontece?

Resposta

Selecionada:



c.

O código não compila, porque as declarações override em Warrior e Mage deixam de corresponder a um membro virtual (ou abstrato) na classe base.

Pergunta 10

0,25 em 0,25 pontos



Em um sistema, existe uma coleção do tipo `IDamageable[]` que contém objetos de classes diferentes, e o código executa `target.TakeDamage(30)` dentro de um `foreach`. Qual alternativa expressa corretamente a divisão entre o que é verificado em tempo de compilação e o que é decidido em tempo de execução, além do motivo de o acoplamento do código chamador ser considerado baixo?

Resposta

Selecionada:

☒ b.

Em compilação, confirma-se que o tipo estático (`IDamageable`) declara `TakeDamage(int)`; em execução, o runtime seleciona a implementação concreta conforme o tipo real do objeto; o baixo acoplamento decorre de o chamador depender do contrato, não da classe concreta.